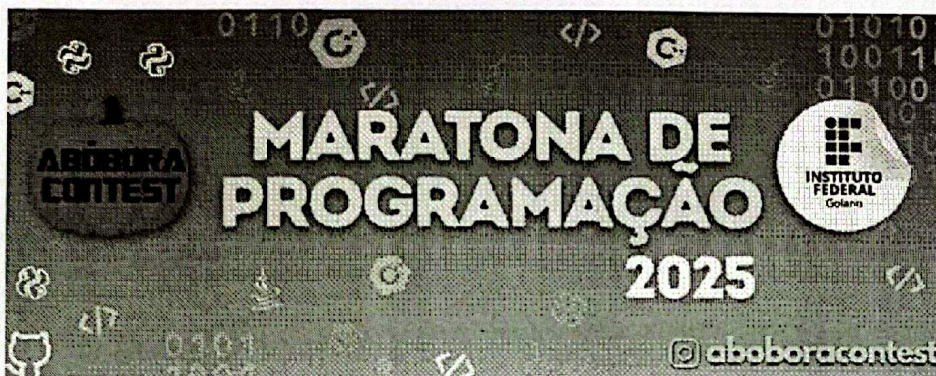




6º Abóbora Contest & 3ª Maratona de Programação IF
30 de Setembro de 2025
Instruções para uso do sistema de submissão Boca

Adaptado de ICPC Latin American Regional Contests, 2024.

1. Esclarecimentos (*clarifications*)

Toda a comunicação com os juízes é feita através das mensagens de **esclarecimentos**. Para solicitar um esclarecimento sobre algum problema você deverá utilizar a interface *web* do Boca:

- (a) Abra seu navegador;
- (b) Efetue login como time;
- (c) Acesse a opção *clarifications*. Escolha o problema sobre o qual você necessita esclarecimento, digite sua pergunta e submeta.

Tanto os pedidos de esclarecimento enviados por vocês quanto as respostas dos juízes estarão disponíveis nessa mesma tela. Haverá um alerta pela interface *web* sempre que você obtiver resposta para algum pedido de esclarecimento. Caso o juiz entenda que o seu esclarecimento se aplica aos demais, será feito um esclarecimento global.

2. Placar (*score board*)

Para visualizar o placar (*score board*), onde você poderá acompanhar o *ranking* entre os times, você deverá utilizar a interface *web* do Boca:

- (a) Abra seu navegador;
- (b) Efetue login como time;
- (c) Acesse a opção *Score*. Você terá acesso ao *ranking* completo.

3. Tarefas (*tasks*)

A aba *Tasks* permite que o time envie códigos para impressão, bem como solicitar ajuda de algum membro do suporte (*staff*).

- (a) Abra seu navegador;
- (b) Efetue login como time e vá até a aba *Tasks*;
- (c) Para imprimir um arquivo selecione ele no disco e clique em enviar (*Send*);
- (d) Para solicitar ajuda, clique no botão *S.O.S.*. Note que a ajuda feita pelo suporte terá relação *apenas* com problemas na máquina ou outro problema físico.

Problema A
Caminho da Colheita

Nome do arquivo: A.c, A.cpp, A.java, ou A.py

Timelimit: 1

Por Athos José

Seu Zé, fazendeiro importante de Rio Verde tem uma plantação de abóboras organizadas em uma área retangular de tamanho $N \times M$, cada muda de abóbora está em uma área 1×1 e contém uma certa quantidade de abóboras.

Costumeiramente, Seu Zé faz a colheita em um percurso único, começando no canto superior esquerdo do campo (posição $(1, 1)$) e termina sempre no canto inferior direito do campo (N, M) . A cada passo, ele mover-se apenas para a direita ou para baixo.

A plantação de Seu Zé cresceu muito e acredita que o carinho que usa para colher não é suficientemente grande para acomodar todas as abóboras no caminho. Agora ele pediu sua ajuda para que dado o campo informa-se qual é máximo de abóboras possível de colher ao longo do melhor caminho.

	1	2	3	4
1	5	2	3	1
2	1	8	1	1
3	4	2	1	10

Figura 1: Exemplo da entrada com células coloridas

Entrada

A primeira linha contém dois inteiros N e M ($1 \leq N, M \leq 2000$), representando respectivamente o número de linhas e colunas da grade.

Em seguida, seguem N linhas, cada uma contendo M inteiros $C_{i,j}$ ($0 \leq C_{i,j} \leq 10^4$), representando a quantidade de abóboras em cada posição i e j .

Saída

Imprima um único inteiro, sendo o número máximo de abóboras que podem ser colhidas seguindo as regras.

Exemplos de Entrada	Exemplos de Saída
3 4 5 2 3 1 1 8 1 1 4 2 1 10	28

Tabela 1: Exemplos de entradas e saídas

Problema B
O Desafio do Chocolate do Zé do Grafo
Nome do arquivo: B.c, B.cpp, B.java, ou B.py
Timelimit: 1

Por André da Cunha Ribeiro

Todo competidor de maratona de programação sabe que as longas horas de um concurso exigem um combustível especial para o cérebro. E qual é o lanche preferido de 10 entre 10 programadores para manter o foco e a energia? Chocolate, é claro! A combinação de açúcar e cafeína parece ter sido feita sob medida para transformar um bug teimoso em um cobiçado "Accepted".

Zé do Grafo, um famoso professor de Teoria dos Grafos e um ávido criador de quebra-cabeças, propôs um desafio para seus alunos. Ele apresentou uma barra de chocolate retangular, dividida em pequenos quadrados por sulcos, formando uma grade de N fileiras e M colunas.

O desafio é o seguinte: para dividir a barra de chocolate em $(N * M)$ quadrados de chocolate individuais, os alunos só podem realizar um tipo de movimento. A cada passo, eles podem pegar um único pedaço de chocolate e quebrá-lo ao longo de um dos sulcos (seja na horizontal ou na vertical), resultando em dois pedaços menores.

Sua tarefa é criar um programa que ajude os alunos do Zé do Grafo a descobrir rapidamente o número total de quebras necessárias para transformar a barra inteira em quadrados individuais.

Entrada A entrada contém na primeira linha o valor $1 \leq X \leq 1000$ que representa o número dos casos de teste. Seguido de X leituras dos valores de N, M ($1 \leq N, M \leq 10^9$) indicando as dimensões dos chocolates.

Saída Para cada caso de teste, o programa deve produzir uma linha na saída, contendo um único inteiro que representa o número total de quebras necessárias para dividir o chocolate daquele caso de teste.

Exemplos de Entrada	Exemplos de Saída
4	3
2 2	23
4 6	55
7 8	836
27 31	

Tabela 2: Exemplos de entradas e saídas

Problema C
Otimização de Rotas de Coleta de Solo - Simple Smart Agro
Nome do arquivo: C.c, C.cpp, C.java, ou C.py
Timelimit: 2

Por Alexandre da Silva

A Simple Smart Agro está desenvolvendo um sistema automatizado para coleta de amostras de solo em uma fazenda. O sistema precisa otimizar as rotas de coleta para maximizar a eficiência do processo. Existem N pontos específicos onde amostras de solo devem ser coletadas. O veículo de coleta parte sempre do depósito localizado na origem $(0,0)$ e deve retornar ao mesmo ponto após coletar todas as amostras. Devido às características do terreno e dos equipamentos, o veículo só pode se mover nas direções cardeais (norte, sul, leste, oeste) e sempre em linha reta entre dois pontos consecutivos da rota. A distância entre dois pontos (x_1, y_1) e (x_2, y_2) é calculada usando a distância Manhattan: $|x_1 - x_2| + |y_1 - y_2|$.

Entrada

- A primeira linha contém um inteiro N ($1 \leq N \leq 10$), representando o número de pontos de coleta;
- As próximas N linhas contêm dois inteiros x_i e y_i ($-1000 \leq x_i, y_i \leq 1000$), representando as coordenadas do i -ésimo ponto de coleta;
- Todos os pontos de coleta são distintos entre si;
- Nenhum ponto de coleta coincide com a origem $(0,0)$.

Saída

- Uma única linha contendo um inteiro: a menor distância total (distância Manhattan) necessária para visitar todos os pontos de coleta e retornar à origem.

Exemplos

Exemplo de Entrada	Exemplo de Saída
3 2 3 7 1 5 6	26
4 3 4 8 2 12 8 6 10	42
1 2 2	8

Explicação do Primeiro Exemplo

Partindo da origem $(0,0)$, temos os pontos de coleta em $(2,3)$, $(7,1)$ e $(5,6)$. Uma rota ótima seria: $(0,0) \rightarrow (2,3) \rightarrow (5,6) \rightarrow (7,1) \rightarrow (0,0)$

Calculando as distâncias:

- $(0,0)$ para $(2,3)$: $|2 - 0| + |3 - 0| = 2 + 3 = 5$
- $(2,3)$ para $(5,6)$: $|5 - 2| + |6 - 3| = 3 + 3 = 6$
- $(5,6)$ para $(7,1)$: $|7 - 5| + |1 - 6| = 2 + 5 = 7$
- $(7,1)$ para $(0,0)$: $|0 - 7| + |0 - 1| = 7 + 1 = 8$

Distância total: $5 + 6 + 7 + 8 = 26$

Problema D
Agência de Espionagem
Nome do arquivo: D.c, D.cpp, D.java, ou D.py
Timelimit: 1
Por Athos José

A Agência Nacional de Espionagem mantém em seus cofres uma coleção de símbolos secretos, representados por uma sequência de letras. Esses símbolos podem ser reorganizados em qualquer ordem, de forma a formar novas mensagens.

Agentes inimigos, porém, estão tentando criar mensagens escondidas dentro dessas reorganizações. Uma mensagem inimiga é considerada possível se puder aparecer como uma sequência contínua de caracteres de alguma permutação dos símbolos guardados no cofre.

Sua missão é, para cada mensagem inimiga recebida, determinar se ela pode ser formada a partir dos símbolos da Agência.

Entrada

A primeira linha contém um inteiro N ($1 \leq N \leq 10^5$), o número de mensagens inimigas a serem analisadas, seguido de uma string S ($1 \leq |S| \leq 10^5$) representando os símbolos secretos do cofre.

Cada uma das próximas N linhas contém uma string Q_i ($1 \leq |Q_i| \leq 10^5$), representando uma mensagem inimiga a ser verificada.

A soma dos comprimentos de todas as mensagens Q_i não excede 10^9 .

Todas as strings são compostas apenas de letras minúsculas do alfabeto inglês (a-z).

Saída

Para cada mensagem Q_i imprima uma linha contendo:

- "SIM", se a mensagem pode ser formada como substring de alguma permutação de S .
- "NAO", caso contrário.

Exemplos de Entrada	Exemplos de Saída
3 abac	SIM
caa	NAO
aaa	SIM
bac	

Tabela 3: Exemplos de entradas e saídas

Problema E
Estória da Estônia
Nome do arquivo: E.c, E.cpp, E.java, ou E.py
Timelimit: 1
Por Márcio Belo

O professor Márcio Belo sempre gostou de charadas matemáticas. Um dia ele se deparou com uma estória, pra lá da Estônia. Dado um número primo ímpar p , sempre é possível um triângulo de lados inteiros x , y e z , em ordem crescente de tamanho com as seguintes propriedades:

- A diferença da soma dos lados menores do triângulo pelo lado maior é igual ao primo p ;
- A diferença da soma das áreas dos quadrados de lados x e y e a área do quadrado de lado z é igual ao primo p .

Em matematiqûês:

Dado um primo $p \geq 3$, sempre existem triplas (x, y, z) , com $x \leq y \leq z$ que satisfazem:

$$\begin{aligned}x + y - z &= p \\ x^2 + y^2 - z^2 &= p\end{aligned}$$

Quem são essas triplas (x, y, z) ? Você pode ajudar o professor?

Entrada Cada entrada contém uma única linha com o número primo p , com $(3 \leq p < 2^{16})$.

Saída O sistema deve retornar todas as triplas (x, y, z) em ordem crescente de x . Uma tripla por linha,

Exemplos de Entrada	Exemplos de Saída
3	(4, 6, 7)
5	(6, 15, 16) (7, 10, 12)
13	(14, 91, 92) (15, 52, 54) (16, 39, 42) (19, 26, 32)
65267	(65268, 2129923278, 2129923279) (97900, 130534, 163167)

Tabela 4: Exemplos de entradas e saídas

Problema F
Monitorando Pragas
Nome do arquivo: F.c, F.cpp, F.java, ou F.py
Timelimit: 1
Por Emanuel Silva Araujo

O agricultor Seu Jayme sofre constantemente com infestações de insetos em sua lavoura de soja. Para reduzir perdas e otimizar o manejo, ele procurou a EMBRAPA para desenvolver uma ferramenta de monitoramento diário de pragas. Sua tarefa como programador é implementar um programa que leia os valores de população diária e identifique situações de risco, emitindo alertas de acordo com os seguintes critérios:

- Alerta por pico diário [Pico]
 - Se em qualquer dia a população registrada ultrapassar 200 insetos.
- Alerta por variação em 7 dias [7 Dias]
 - Se a diferença percentual entre o valor mínimo e o máximo em um intervalo de 7 dias consecutivos for maior ou igual a 20%.
- Alerta por crescimento contínuo [Crescimento]
 - Se ocorrerem 4 dias seguidos de crescimento, e em cada dia a população for pelo menos 5% maior que a do dia anterior.

Entrada

Um número inteiro ($0 \leq N \leq 360$), representando a quantidade de dias monitorados (entre os dias 0 e 360). Uma sequência de N números inteiros ($0 < X \leq 5000$), cada um representando a estimativa da população de insetos no respectivo dia.

Saída

Para cada alerta identificado, o programa deve imprimir:

- O tipo do alerta;
- O(s) dia(s) correspondente(s);
- O valor percentual (quando aplicável), com duas casas decimais.

Caso não tenha nenhum alerta, deve imprimir uma mensagem de "Nenhum Alerta". Para dias que tiveram mais de um alerta a ordem de impressão é: [Pico] > [7Dias] > [Crescimento].

Exemplos de Entrada	Exemplos de Saída
10 50 60 70 80 90 200 210 220 180 170	[Crescimento] 1-4: maior que $\geq 5\%$ [Crescimento] 1-5: maior que $\geq 5\%$ [Crescimento] 1-6: maior que $\geq 5\%$ [Pico] 7: 210 insetos [7 Dias] 1-7: 320.00% [Crescimento] 1-7: maior que $\geq 5\%$ [Pico] 8: 220 insetos [7 Dias] 2-8: 266.67% [7 Dias] 3-9: 214.29% [7 Dias] 4-10: 175.00%
9 100 105 112 118 90 96 100 120 80	[Crescimento] 1-4: maior que $\geq 5\%$ [7 Dias] 2-8: 33.33%
8 118 120 118 140 140 140 130 100	Nenhum Alerta

Tabela 5: Exemplos de entradas e saídas

Problema G
Comportamento Dinâmico
Nome do arquivo: G.c, G.cpp, G.java, ou G.py
Timelimit: 1

Por Heverton Barros de Macêdo

Durante uma investigação sobre o comportamento de sistemas dinâmicos, Wolfram definiu o seguinte cenário para efetuar experimentos: dada uma cadeia de N células, linearmente distribuídas, cada célula armazena um único estado (bit), e além disso as células das extremidades são vizinhas entre si. Para cada instante de tempo t a evolução dessa cadeia ocorre simultaneamente em todas as células, considerando uma regra de evolução previamente definida. Essa regra é aplicada a cada vizinhança de $K = 3$ células gerando um novo estado, conforme indicado na Tabela 6.

Tabela 6: Regra aplicada para evolução do experimento de Wolfram.

Regra para vizinhança	000	001	010	011	100	101	110	111
Novo estado para célula central	1	0	0	1	0	0	0	1

Como exemplo, considere um experimento (veja a Tabela 7) com a cadeia de bits formada por 11 células ($N = 11$), em que no instante $t = 0$ apenas um bit, exatamente no centro da cadeia, possui o estado 1 e as demais células possuem o estado 0, evoluindo por 4 passos de tempo ($t = 4$).

A Tabela 7 ilustra o experimento com a configuração inicial ($t = 0$) formada por: 00000100000. No próximo passo de tempo ($t = 1$), todas as células evoluem (podem alterar o estado) considerando a vizinhança indicada na regra (Tabela 6). Em destaque na cor vermelha (Tabelas 6 e 7) está a atualização da célula central, formada pela vizinhança 010 que, segundo a regra, deve evoluir no próximo passo de tempo ($t = 1$) para o estado 0 (destaque na cor verde). É possível notar no exemplo, a evolução da cadeia inicial por 4 passos de tempo. Vale destacar que as células na extremidade (esquerda e direita) estão conectadas (formato de anel - toroidal).

Tabela 7: Exemplo de um experimento por 4 passos de tempo.

Tempo	Cadeia linear de N células										
$t = 0$	0	0	0	0	0	1	0	0	0	0	0
$t = 1$	1	1	1	1	0	0	0	1	1	1	1
$t = 2$	1	1	1	0	1	0	0	1	1	1	1
$t = 3$	1	1	0	0	0	0	0	1	1	1	1
$t = 4$	1	0	0	1	1	1	0	1	1	1	1

Sua missão é ajudar Wolfram a obter a configuração dos estados após t passos de tempo, considerando uma sequência inicial qualquer no tempo $t = 0$ e a aplicação da regra indicada no enunciado.

Entrada

A entrada contém vários casos de teste. Cada linha contém um inteiro t ($0 < t \leq 1000$), que representa a quantidade de passos de evolução, seguido pela cadeia de bits no tempo $t = 0$, que pode variar conforme a quantidade de células da amostra N ($3 \leq N < 100$).

Saída

O programa deve produzir como saída a configuração da cadeia no espaço linear de células após a evolução de t passos de tempo.

Exemplos de Entrada	Exemplos de Saída
1 00000100000	11110001111
2 00000100000	11100101111
3 00000100000	11000001111
578 00000100000	00000111101
978 011001111000110111010011	100111000001110000011101

Tabela 8: Exemplos de entradas e saídas